

Multimodal RAG

As-Built Architecture and Operations Guide

Project	E:\Rag_App\multimodal-rag
Purpose	Answer questions over complex PDFs, returning synthesized text plus the exact retrieved images
Stack	PyMuPDF - Gemini - Groq - Qdrant - FastAPI - Streamlit
Cost	Free tier throughout: no paid API, no GPU, no cloud
Status	Built, running, and measured against a live document
Note	This supersedes the earlier design document, which described a Claude / Voyage stack that was replaced before implementation

Contents

- 1 What was built
- 2 System architecture
- 3 Ingestion - the two routes
- 4 Storage - what a record looks like
- 5 Retrieval - hybrid search
- 6 Answering - the fallback chain
- 7 Diagram generation
- 8 Module reference
- 9 Measured results
- 10 Operations runbook
- 11 Free-tier engineering
- 12 Known limitations and next steps

1. What was built

A retrieval-augmented generation pipeline that ingests PDFs containing text, diagrams, screenshots and tables, and answers questions about them by returning **both** a synthesized text answer **and** the exact image files the answer relies on.

The idea the whole system turns on

A picture cannot be searched. So during ingestion a vision model writes text about every figure, and that text is what gets indexed. At answer time the description is thrown away and the real PNG is handed to the model. Descriptions get you TO the image; the image itself is what gets read and returned.

Verified against a real document

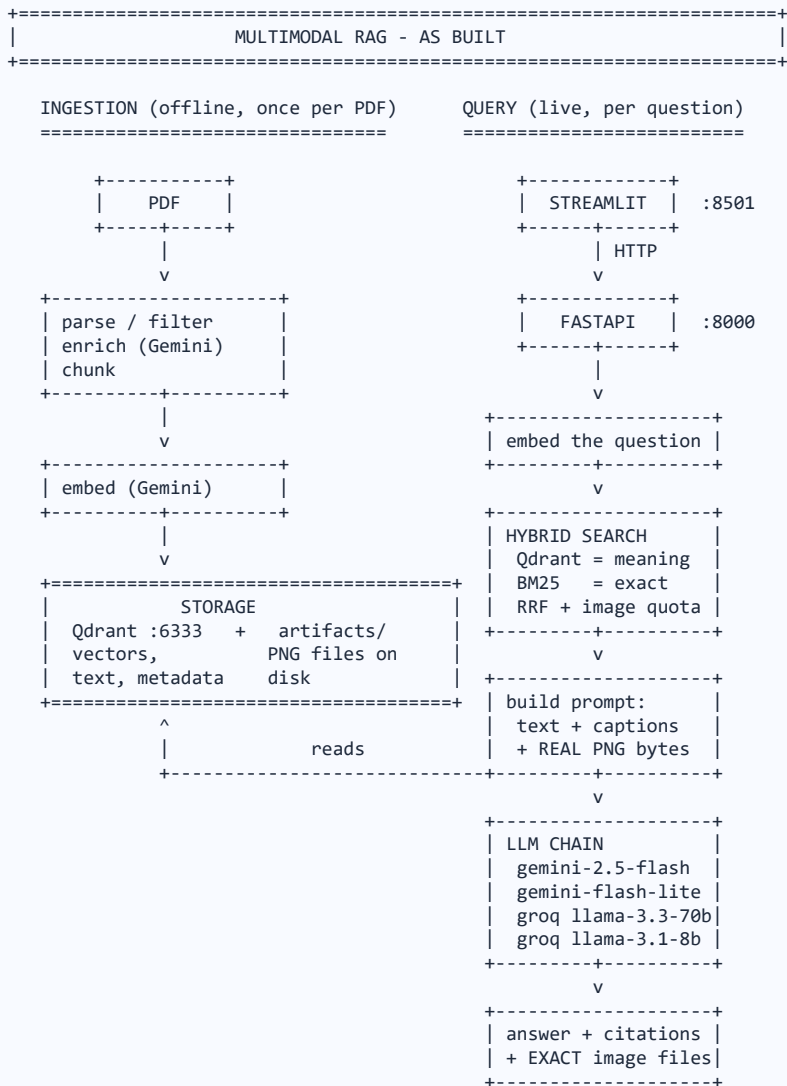
Test document	Operating_AI-agents.pdf - 11 pages, design-heavy, no extractable text layer
Result	48 searchable chunks: 37 text, 11 images
Retrieval quality	hit@5 = 100%, MRR 0.809 over a 40-question golden set
Image retrieval	100% - every question whose answer was a figure found it
Cost to build and run	Zero

Final stack

Layer	Choice	Why
PDF parsing	PyMuPDF	Text with coordinates, image extraction, table detection and page rendering from one library
Vision / enrichment	Gemini 2.5 Flash	Free multimodal; transcribes pages and describes figures
Embeddings	gemini-embedding-001, 768 dims	Free; removes a whole vendor from the design
Vector store	Qdrant 1.18 in Docker	Payload filtering drives the image quota; dashboard makes debugging visual
Keyword search	rank-bm25, in process	Catches exact identifiers that embeddings blur
Answering	Gemini, falling back to Groq	Four-model chain; survives rate limits
Diagram generation	Graphviz DOT via the same chain	Free, faithful, editable - no image model needed
API	FastAPI	Thin REST layer; also serves the PNGs
UI	Streamlit	Chat with images rendered inline

2. System architecture

Two pipelines, joined only by storage

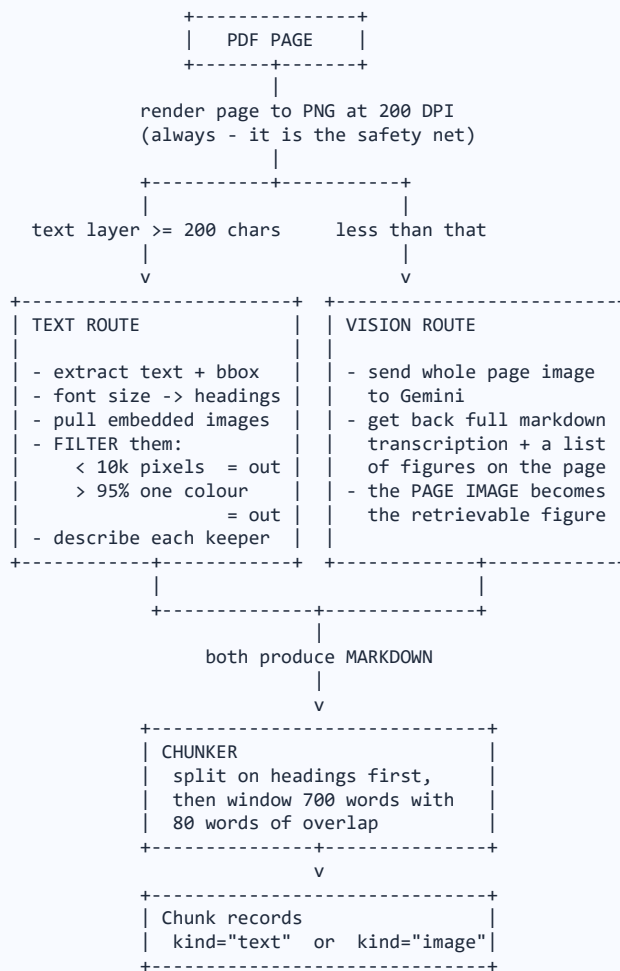


Ingestion never calls retrieval. The two halves communicate only through storage, which means the index can be rebuilt without re-parsing a PDF, and retrieval can be tuned without re-spending vision quota.

3. Ingestion - the two routes

Every page is inspected and sent down one of two routes automatically. The test document happened to have no text layer at all, so all 11 pages took the vision route - but both are implemented and either can occur within one file.

Route selection and convergence



	Text route	Vision route
Trigger	page has 200+ characters of extractable text	less than that - a scan or an exported design
Text source	PyMuPDF extraction, headings inferred from font size	Gemini transcribes the whole page to markdown
Figures	embedded images extracted individually, then described	the page render itself is the figure
Cost	one vision call per surviving image	one vision call per page
Output	text chunks + one image chunk per figure	text chunks + one image chunk for the page

Why every page is rendered, even text pages

Some diagrams are not stored as images at all - they are vector drawing instructions, so image extraction finds nothing. The page render always contains them. It costs one PNG per page and removes an entire class of silent failure.

The image filters

Filtering matters more than extraction. A design-heavy PDF can yield hundreds of embedded images, most of them the same logo repeated in every header. Two cheap tests remove them:

- **Size** - under 10,000 pixels total is decoration, not content.
- **Colour variance** - if over 95% of pixels are a single colour it is a rule, a fill or a bullet. Measured on a 64x64 thumbnail, so it costs nothing.

4. Storage - what a record looks like

There is deliberately only **one** record type. A paragraph, a diagram and a table are all a Chunk; they differ only by **kind** and whether they carry an **image_path**. One collection, one code path, and both modalities can be ranked against each other in a single query.

A single Qdrant point

```

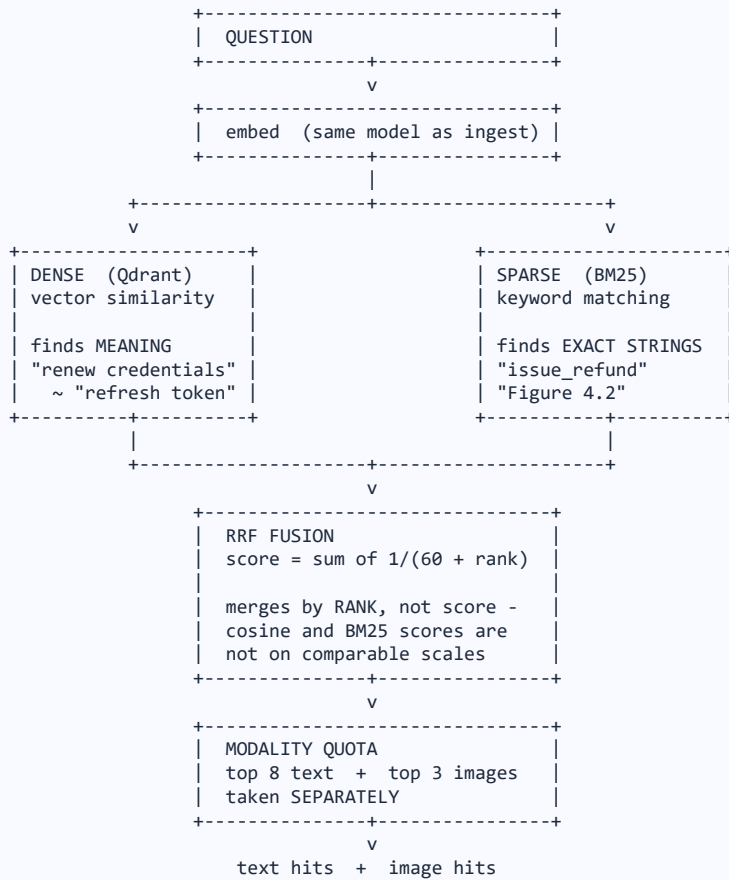
QDRANT POINT (kind = "image")
+-----+
| id      "operating-ai-agents:p10:i0" |
| VECTOR  [0.021, -0.334, 0.912, ...] 768 dims, cosine --> dense search |
| PAYLOAD |
| +- text      "Shipping an AI Agent Application: From |
| |            Prototype to Production. Flow with two |
| |            stages... plus a comparison table..." --> what is searched |
| +- kind      "image"                               --> the modality quota |
| +- page      10                                     --> citation |
| +- heading   "Shipping an AI Agent Application" |
| +- doc_id    "operating-ai-agents" |
| +- image_path "artifacts/operating-ai-agents/ |
| |            page_010.png" -----+ --> loaded at answer time |
+-----+-----+
|
| ACTUAL PNG BYTES |
| on disk, untouched |
+-----+

```

The PNG never enters the database. Qdrant stores a path; FastAPI serves the file. `kind` and `doc_id` are indexed payload fields, which is what makes the modality quota a fast server-side filter rather than a Python loop.

5. Retrieval - hybrid search

Two searches, fused by rank



Why the quota exists

Without it, text chunks win on raw score almost every time and images never reach the model - which would defeat the entire purpose of the project. Taking the best N of each kind separately guarantees figures always get seats at the table.

Every hit records **found_by**: **dense**, **bm25**, or **dense+bm25**. That single field is the debugging handle when a result looks wrong - it tells you immediately whether the problem is in the embeddings or the keyword index.

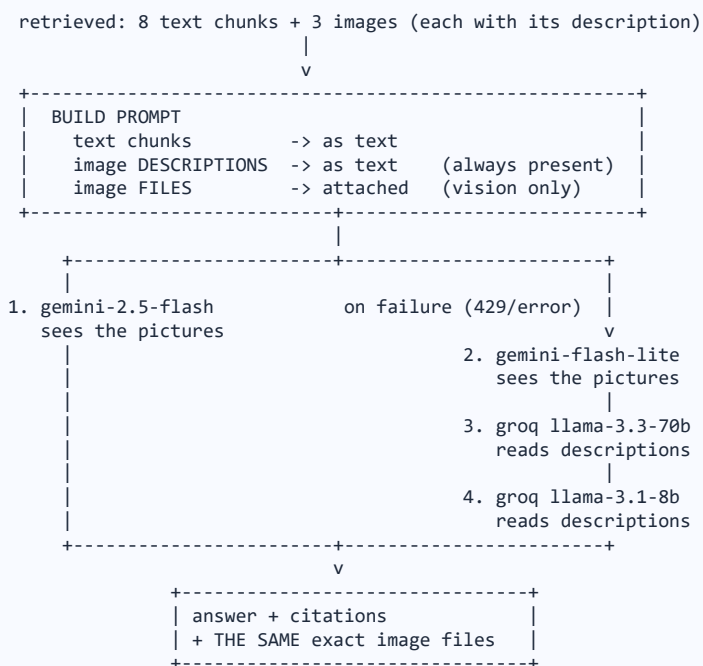
6. Answering - the fallback chain

Four models are tried in order until one succeeds. Two of them cannot see images at all, which shapes the single most important rule in the prompt builder:

The prompt must stand alone without the images

Every figure's written description is included in the prompt text. The real PNGs are attached as a bonus for models that can see. So when the chain falls through to a text-only model, the answer is still grounded - it reads descriptions instead of pixels. Quality drops; nothing breaks. And the user receives the same exact image files either way, because retrieval already chose them.

Graceful degradation



A real gap this exposed

Gemini enforces the response schema; Groq's JSON mode only guarantees that the output is valid JSON, not that it matches your schema. The Groq path therefore returned an empty `used_pages` list. It is backfilled from whichever figures the model did cite - that page genuinely is a source, so this is inference, not invention. Verified working on the weakest model in the chain.

General lesson

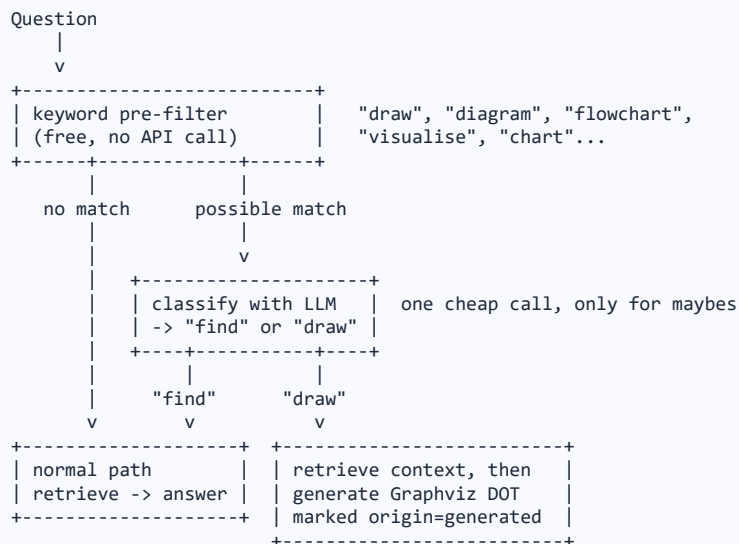
A structured-output schema is a contract only where the provider enforces it. Treat schema compliance as best-effort on any provider whose JSON mode is not schema-aware, and validate on your side.

7. Diagram generation

Retrieval finds figures that exist. A separate branch **draws new ones** when asked. It does so by generating Graphviz DOT source rather than calling an image model.

	Diagram-as-code (built)	Text-to-image (not built)
Cost	Free - it is text generation	Billed on every provider checked
Faithfulness	Labels are exact strings from the document	Model paints text; can garble or invent it
Editable	Yes - fix one node in the source	No - regenerate the whole picture
Install	None; Streamlit renders DOT natively	None, but needs billing enabled
Scope	Flows, hierarchies, sequences, state machines	Anything visual

The intent branch



The keyword pre-filter means an ordinary question pays **nothing** for intent detection - no API call is made unless the wording actually looks like a drawing request.

Generated output is labelled, always

Anything drawn carries origin="generated" and the UI marks it clearly. A retrieved image is evidence from the document; a generated diagram is the model's assertion. Allowing those two to be confused would quietly destroy the guarantee the whole system exists to provide.

8. Module reference

File	Responsibility
app/config.py	Every setting in one place: keys, model chain, thresholds, paths
app/llm.py	ask(), ask_json(), embed(), embed_query() - the only file that talks to a model provider
app/models.py	Chunk dataclass plus the pydantic schemas the vision model fills in
app/enrich.py	Page transcription and figure description, with a JSON cache
app/ingest.py	Route selection, image filters, markdown conversion, chunking
app/store.py	Qdrant collection lifecycle, batched embed and upsert, scroll, filtered search
app/retrieve.py	BM25 index, RRF fusion, modality quota
app/answer.py	Prompt assembly with real PNGs, response shaping
app/diagram.py	Intent routing and Graphviz generation
api.py	FastAPI: /health, /query, /ingest, /documents, /artifacts
ui.py	Streamlit chat, inline images, retrieval detail panel
run_ingest.py	CLI: PDF -> artifacts/
run_index.py	CLI: artifacts/ -> Qdrant
eval/make_golden.py	Generate a starter question set
eval/run_eval.py	Score retrieval; no LLM required

A rule worth keeping

api.py and ui.py contain no pipeline logic - they only call into app/. That is why the entire system can be driven and tested from a Python prompt without starting either server.

9. Measured results

Retrieval quality

Scored over a 40-question golden set at k=5. No LLM is involved in scoring - the expected page for each question is known, so the measurement is deterministic, free, and runs in seconds.

Retriever	hit@5	MRR	Image questions
Dense only	97.5%	0.829	100%
BM25 only	92.5%	0.608	90%
Hybrid (in use)	100%	0.809	100%

Read this table honestly

Hybrid achieves perfect recall, but its MRR is slightly LOWER than dense alone - blending BM25's weaker ranking occasionally pushes a dense top-1 down a place. That is a good trade when eight chunks are sent to the model, because recall is what matters there. It is also precisely the gap a reranker would close.

And a caveat on the numbers

The golden questions were generated from the chunks themselves, so their wording leaks in and the scores are optimistic. They are a baseline to detect regressions, not proof of real-world accuracy. Rewriting the questions by hand is where the real signal comes from.

Behaviour spot-checks

Query type	Question asked	Result
Semantic	How should tools be defined so they fail predictably?	Top hit: 3.2 Tool contracts and boundaries - correct, found on meaning alone
Exact identifier	issue_refund ORD pattern maximum 5000	Top hit: 7.1 Tool contract as a strict schema - a string that existed only as pixels in a code screenshot
Visual	diagram of the stages from prototype to production	Page 10 figure returned, out-ranking every text chunk
Fallback forced	same question, Groq only, no vision	Grounded answer from descriptions; identical image files returned
Drawing	Draw a flowchart of the stages from prototype to production	Valid clustered Graphviz DOT generated from pages 1, 7 and 10

10. Operations runbook

Start everything

```
cd E:\Rag_App\multimodal-rag

docker compose up -d                # Qdrant on :6333
.\.venv\Scripts\uvicorn.exe api:app --reload  # API on :8000
.\.venv\Scripts\streamlit.exe run ui.py      # UI on :8501
```

Add a document

```
py run_ingest.py data/YourDoc.pdf      # parse + enrich -> artifacts/
py run_index.py                        # embed + load into Qdrant

# or just drag the PDF into the Streamlit sidebar
```

Measure retrieval

```
py eval/make_golden.py 40      # starter question set (edit it by hand)
py eval/run_eval.py           # score current settings
py eval/run_eval.py --compare  # dense vs bm25 vs hybrid
```

Useful endpoints

URL	What it gives you
http://localhost:8501	The application
http://localhost:8000/docs	Interactive API documentation
http://localhost:8000/health	Qdrant reachability, point count, model chain
http://localhost:6333/dashboard	Browse vectors and payloads directly

After a reboot

Only `docker compose up -d` is needed to bring the data back. The Qdrant volume persists on disk - a clean container shutdown loses nothing, which was verified after an unplanned 12-hour stop: all 48 points survived.

11. Free-tier engineering

Running entirely on free tiers is not just a matter of choosing free models. Three mechanisms make it actually work:

Throttling

Ingestion sleeps between vision calls, keeping request rate under the free-tier ceiling. Configurable via `GEMINI_SLEEP_SECONDS`; the default of 4.5 seconds corresponds to roughly 13 requests per minute.

Caching

Every vision result is written to `artifacts/<doc_id>/enriched.json` keyed by page or image. Re-running an ingest costs nothing and repeats no call. This is what makes it safe to iterate on chunking or filters without re-spending quota.

The chain itself

During generation of the golden set, Gemini 2.5 Flash hit its rate limit part-way through. The chain fell through to the next model automatically and the run completed without intervention. The fallback design is not theoretical - it was exercised by a real limit during development.

Cost control that is already in place

- Images are downscaled to 1280px before being attached to a prompt. A 200 DPI page render is roughly 1700x2200; sending it raw burns quota for no readability gain.
- Text chunks are trimmed to 1800 characters in the prompt.
- The intent pre-filter avoids an API call on every ordinary question.
- Retrieval scoring uses no LLM at all, so evaluation is unlimited and instant.

12. Known limitations and next steps

Limitations

Limitation	Detail
No reranker	Free API rerankers do not exist. Hybrid search alone is good but leaves MRR on the table.
Golden set is synthetic	Questions were generated from the chunks, so scores are optimistic.
No text-to-image	Only structural diagrams can be drawn. Illustration needs a billed image model.
Single-user assumptions	Ingest jobs are held in memory; BM25 is rebuilt at process start.
Vision cost scales per page	A document with no text layer costs one vision call per page. Large scanned documents will hit daily limits.
No OCR fallback	If the vision model is unavailable there is no local transcription path.

Natural next steps, in order of value

- 1 **Hand-write the golden questions.** The single highest-value improvement. Replaces optimistic numbers with trustworthy ones and makes every later change measurable.
- 2 **Add a local reranker** if MRR matters - a small cross-encoder run locally would close the precision gap without any API cost.
- 3 **Persist the BM25 index** rather than rebuilding it at startup, once the corpus grows beyond a few thousand chunks.
- 4 **Answer-quality evaluation** using an LLM judge over the golden set, to measure faithfulness rather than only retrieval.
- 5 **Text-to-image generation**, if illustration is ever needed - the branch is already structured to accept it, and only the generation function would change.

The rule that matters most going forward

Extraction and description quality cap everything downstream. No amount of retrieval tuning rescues a badly filtered, badly captioned index - so when results disappoint, look at artifacts/ and enriched.json before touching the search code.